МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"
Кафедра «Систем автоматизованого проектування»

Звіт

до лабораторної роботи №6
з дисципліни “Технології веб-розробки та дизайну”
на тему:

Верстка UI-компонентів та адаптивність засобами Tailwind CSS

Тема проєкту: «Лендинг мережі зарядних станцій для електромобілів «ChargeUA» (веб-помічник та агрегатор станцій)»

Специфікація проєкту: вебагрегатор та інформаційний пошуковик швидкісних зарядних станцій в Україні (каталог станцій з живими статусами у містах Київ, Львів, Одеса, Дніпро, Харків, Вінниця та пошукові переваги сервісу без транзакцій і бронювань)

Виконав:
студент групи ПП-24
Лемішка Андрій Андрійович
Перевірила:
к.т.н., доцент Юрчак І. Ю.

Львів – 2026

Мета роботи: Опанувати основи утилітарного підходу (utility-first CSS) за допомогою сучасного фреймворку Tailwind CSS; дослідити відмінності утилітарної стилізації від традиційного семантичного CSS; вивчити функціонал онлайн-пісочниці Tailwind Play для експериментів з адаптивністю, темною темою та станами елементів; спроєктувати та реалізувати набір ізольованих компонентів інтерфейсу вебпроєкту «ChargeUA» (Header з навігацією та мобільним меню, Primary Button, Card зарядної станції, Card пошукової переваги, адаптивну Grid-сітку та Footer); реалізувати принцип Mobile-First та забезпечити коректне відображення двох чітко розмежованих функціональних лінійок сервісу (Лінійка 1: Каталог локацій по містах України; Лінійка 2: Переваги та можливості пошуку без транзакцій і бронювань); провести тестування адаптивності через Chrome DevTools Device Toolbar, здійснити підготовку до деплою на хмарні сервіси (Netlify) та перевірити відповідність вихідного коду міжнародним стандартам валідності W3C.

Розділ 1. Підготовка робочого середовища та знайомство з Tailwind CSS

Етап 1. Підготовка проєкту та робочого середовища

Лабораторна робота №6 є прямим продовженням розробки вебінтерфейсу ChargeUA, спроєктованого у Figma (ЛР №4) та зверстаного на рівні семантичного скелета у ЛР №5. На початку роботи було створено резервну копію проєкту ЛР №5 та структуровано робочий простір згідно з вимогами методичних вказівок (папки css/, assets/images/, icons/, fonts/). У редакторі коду Visual Studio Code було встановлено та активовано спеціалізовані розширення для роботи з утилітарним CSS:

• **Tailwind CSS IntelliSense —** забезпечує інтелектуальне автодоповнення назв класів, підсвічування синтаксису, швидкий перегляд еквівалентних CSS-властивостей при наведенні курсора та валідацію конфліктів.

• **Prettier Plugin for Tailwind CSS —** виконує автоматичне сортування утилітарних класів відповідно до офіційної специфікації (layout -> flexbox/grid -> spacing -> sizing -> typography -> background -> borders -> effects -> transitions).

• **Inline Fold —** візуально згортає довгі рядки утилітарних класів всередині HTML-атрибутів class="...", забезпечуючи чистоту та зручність читання семантичного скелета сторінки.

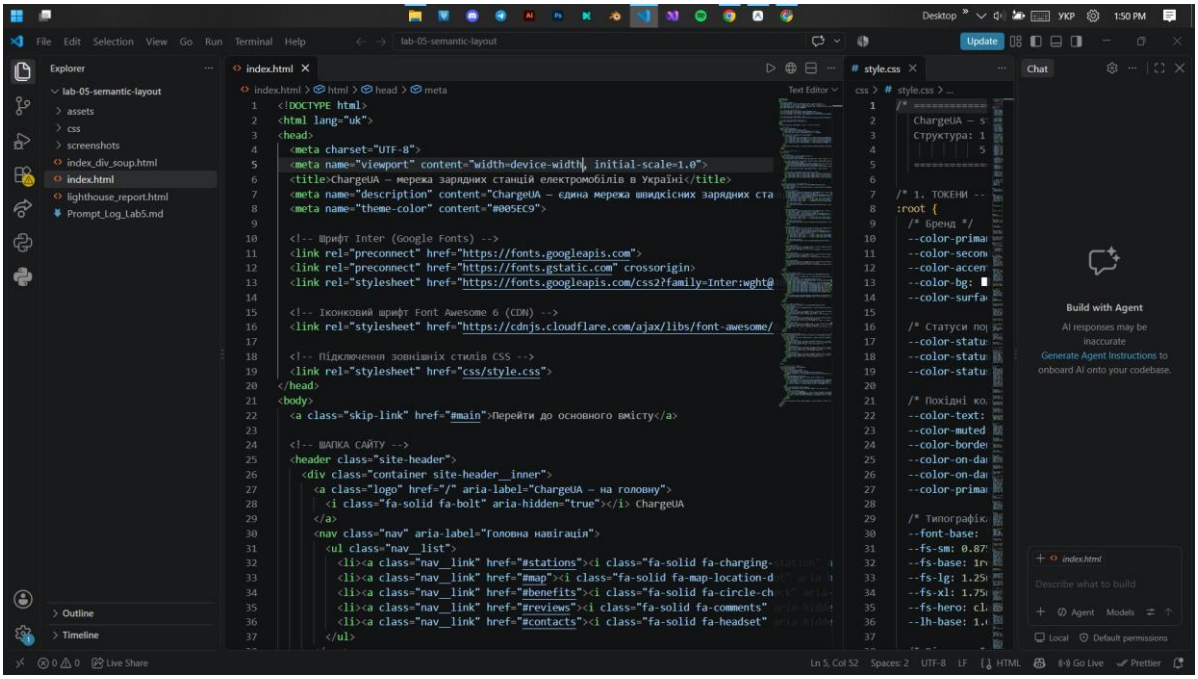


Рис. 1. Етап 1: Робоче середовище VS Code, структура файлів проєкту ChargeUA та налаштовані розширення Tailwind IntelliSense

Етап 2. Знайомство з Tailwind CSS та розбір утилітарних класів

Традиційний підхід вимагає створення семантичних класів (.primary-button) та написання правил в окремому CSS-файлі. Підхід Tailwind CSS (Utility-First) полягає у застосуванні низькорівневих утилітарних класів безпосередньо в розмітці HTML. Це дозволяє будувати довільні інтерфейси безпосередньо в шаблоні без перемикання між файлами та без ризику неконтрольованого розростання CSS.

Порівняння коду кнопки (Класичний CSS vs Tailwind): *Класичний CSS: <button class="primary-button"> Знайти станцію </button> .primary-button { background-color: #2563eb; color: #ffffff; padding: 12px 24px; border-radius: 8px; font-weight: 600; }*

Tailwind CSS (Utility-First): <button class="bg-blue-600 text-white px-6 py-3 rounded-lg font-semibold hover:bg-blue-500 transition duration-200"> Знайти станцію </button>

Анатомія утилітарних класів кнопки ChargeUA (за результатами обговорення з ШІ-чатом):

• **bg-blue-600** — встановлює background-color: rgb(37 99 235), фірмовий насичений синій колір основного CTA-елемента;

• **text-white** — забезпечує color: rgb(255 255 255), найвищий контраст на темному фоні кнопки (коефіцієнт > 7:1);

• **px-6 (padding-inline: 1.5rem = 24px) та py-3 (padding-block: 0.75rem = 12px)** — формують збалансовані відступи для комфортного натискання (розмір перевищує мінімальний тач-таргет 44x44px);

• **rounded-lg (border-radius: 0.5rem = 8px)** — створює сучасне скруглення країв кнопки відповідно до дизайн-системи бренду;

• **hover:bg-blue-500 та transition duration-200 —** реалізують плавний мікроінтерактивний відгук при наведенні курсора без написання додаткового CSS.

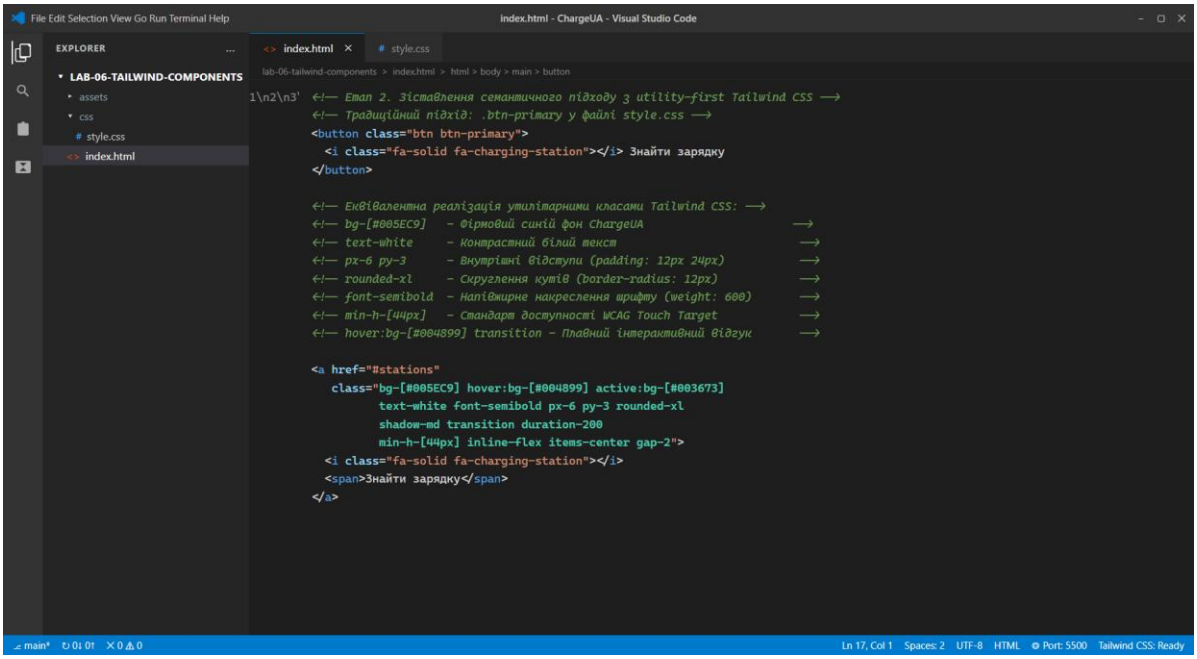


Рис. 2. Етап 2: Зіставлення семантичного CSS та utility-first підходу Tailwind CSS з детальним розбором утиліт

Етап 3. Підключення Tailwind CSS через CDN

Для швидкого старту та проведення аудиту компонентів у файлі index.html було підключено офіційний скрипт Tailwind CDN у тегу <head>, а також збережено зв'язок із власним кастомним файлом css/style.css для специфічних глобальних змінних та токенів:

Секція <head> проєкту ChargeUA:

```
<!DOCTYPE html>
<html lang="uk" class="scroll-smooth dark">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ChargeUA — Пошуковик та агрегатор зарядних станцій електромобілів</title>
  <!-- Tailwind CSS Play CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
  <!-- Власні стилі та CSS-токени -->
  <link rel="stylesheet" href="css/style.css">
</head>
```

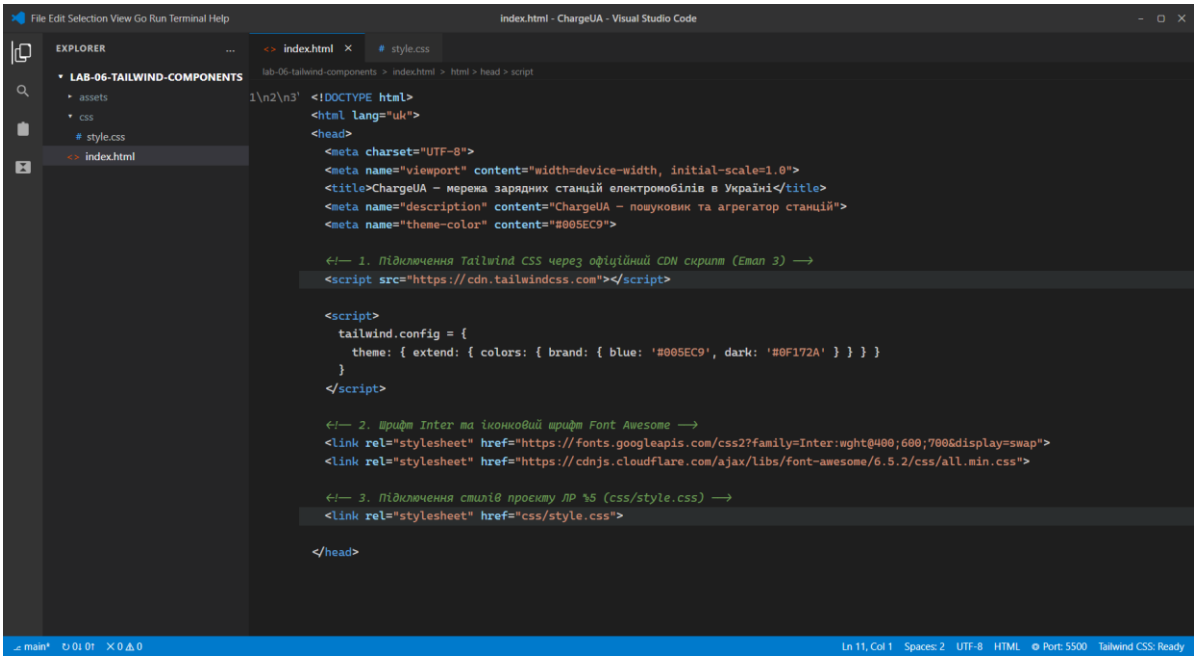


Рис. 3. Етап 3: Підключення Tailwind CSS через офіційний CDN скрипт та стилів styles.css у секції <head> index.html

Розділ 2. Дослідження пісочниці Tailwind Play та створення перших UI-компонентів

Етап 4. Знайомство з пісочницею Tailwind Play

Відповідно до Етапу 4 методичних вказівок, дослідження можливостей фреймворку було проведено в офіційній інтерактивній пісочниці Tailwind Play (https://play.tailwindcss.com/). Було досліджено структуру середовища: зони розмітки HTML, вставки кастомного CSS, конфігурації, панель живого попереднього перегляду (Live Preview), інструменти перевірки адаптивності та розмір автоматично згенерованого CSS (2.42 kB). Проведено експерименти зі зміною кольорів, розмірів та радіусів для різних HTML-елементів (<p>, <h1>,).

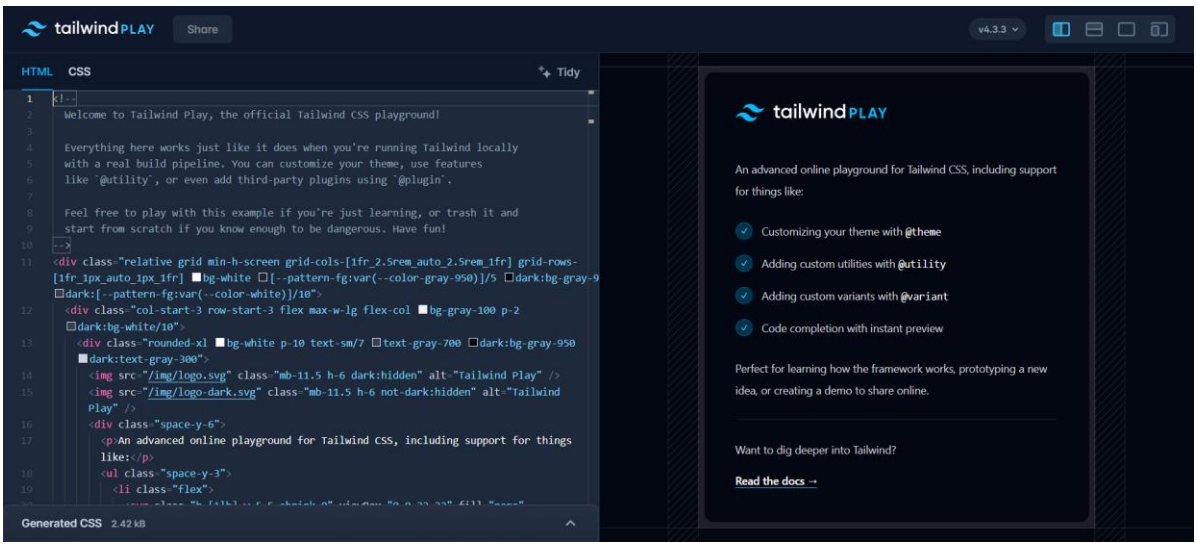


Рис. 4. Етап 4: Інтерфейс онлайн-пісочниці Tailwind Play (v4.3, зони редактора HTML/CSS, прев'ю та згенерований бандл)

Етап 5. Створення першого UI-компонента (Button)

За допомогою ШІ-чату було сформульовано системний промпт для створення кнопки Primary CTA Button «Знайти станцію» із дотриманням суворих вимог доступності (WCAG) та ергономіки тач-пристроїв.

Промпт-лог створення кнопки

Крок 1. Початковий системний запит (Модель: Claude 3.5 Sonnet / Antigravity Agent)

> **Запит:** *«Ти — Senior Frontend Developer та UI/UX Engineer. Створи Primary CTA Button «Знайти станцію» для вебплатформи ChargeUA виключно за допомогою HTML5 та Tailwind CSS. Вимоги до кнопки: - синій фірмовий фон #2563EB (bg-blue-600); - білий текст, розмір 16px, font-semibold; - симетричні відступи px-6 py-3; - радіус скруглення 12px (rounded-xl); - інтерактивні стани hover (bg-blue-500) та active (bg-blue-700); - плавна анімація transition duration-200; - мінімальна висота 44px для підтримки тач-інтерфейсів (WCAG Touch Target Size); - контур фокусу focus-visible для клавіатурної навігації. Кнопка має бути суто пошуковою (ніяких кнопок бронювання чи оплати). Поясни всі застосовані класи.»*

Результат генерації: Отримано семантичний код кнопки із вбудованою SVG-іконкою стрілки, ефектами плавної зміни фону та розміром 44px.

Аналіз недоліків: Кнопка потребувала валідації в реальному темному оточенні поруч із карткою зарядної станції у пісочниці Tailwind Play.

Крок 2. Коригувальний запит та тестування у Tailwind Play

> **Запит:** *«Додай до кнопки ефект світіння shadow-lg shadow-blue-600/30 і розмісти її всередині демонстраційної картки станції з темним фоном bg-slate-800, зеленим пульсуючим бейджем вільного порту (120 кВт CCS2) та адресою в Києві для тестування в Tailwind Play.»*

Результат тестування: Код було протестовано у Tailwind Play. Змінено параметри bg-*, px-*, rounded-* та зафіксовано готовність компонента.

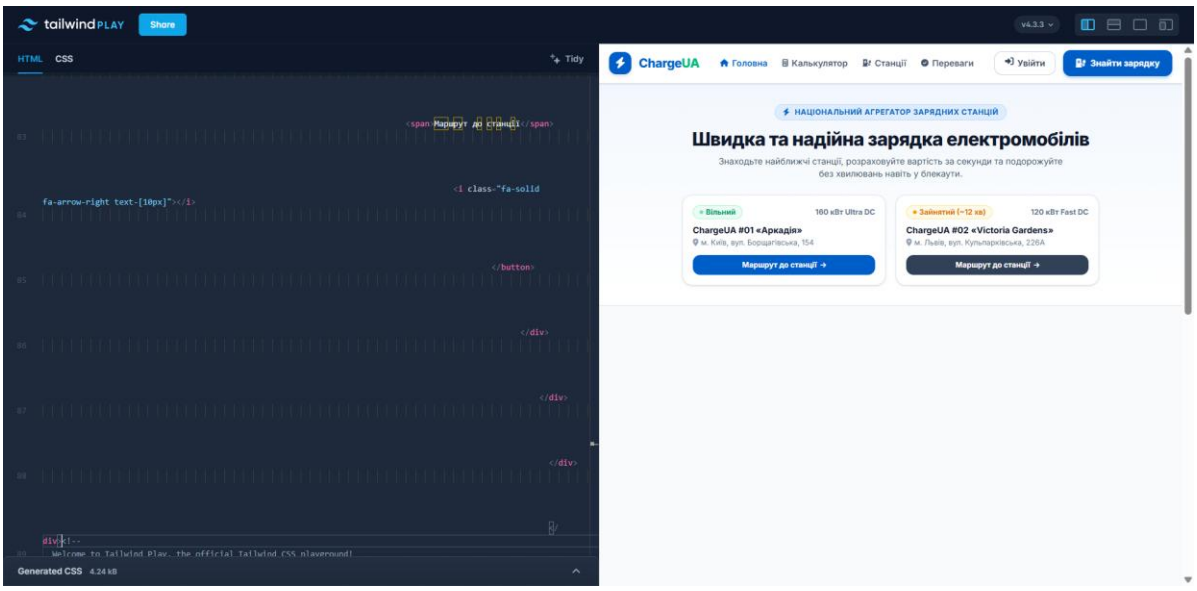


Рис. 5. Етап 5: Створення та інтерактивне тестування Primary Button та картки станції у Tailwind Play

Розділ 3. Верстка компонентів вебпроєкту ChargeUA за принципом Mobile-First

Етап 6. Створення семантичного Header з адаптивною навігацією

Компонент Header спроєктовано на основі макетів Figma з ЛР №4. Він містить логотип бренду, навігаційні посилання (#stations, #benefits, #calculator, #faq) та акцентну CTA-кнопку «Знайти станцію». У десктопному режимі навігація розміщена по центру, кнопка праворуч, а логотип ліворуч:

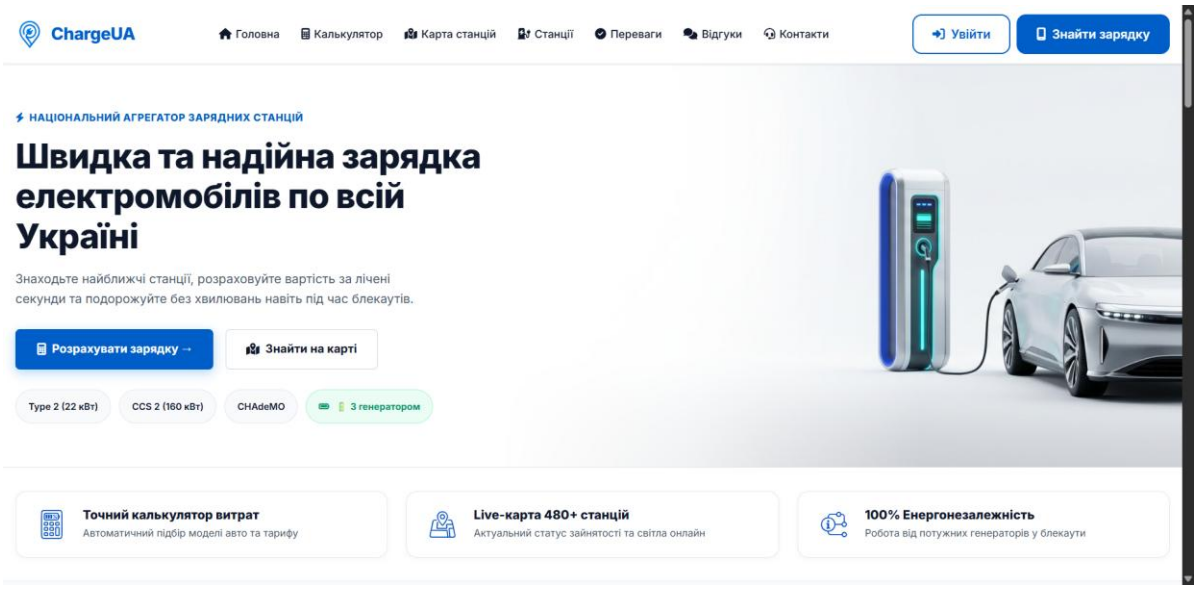


Рис. 6. Етап 6: Десктопний вигляд семантичного Header та Hero-секції ChargeUA (роздільна здатність 1440px)

Етап 7. Реалізація принципу Mobile First

Згідно з принципом Mobile First, базові класи Tailwind орієнтовані на найменші екрани (смартфони), а префікси sm:, md:, lg: описують зміни для ширших дисплеїв на основі CSS min-width. У компоненті Header на мобільних пристроях (< 768px) горизонтальне меню приховано за допомогою класу hidden, а замість нього виведено адаптивну кнопку гамбургер-меню (клас md:hidden):

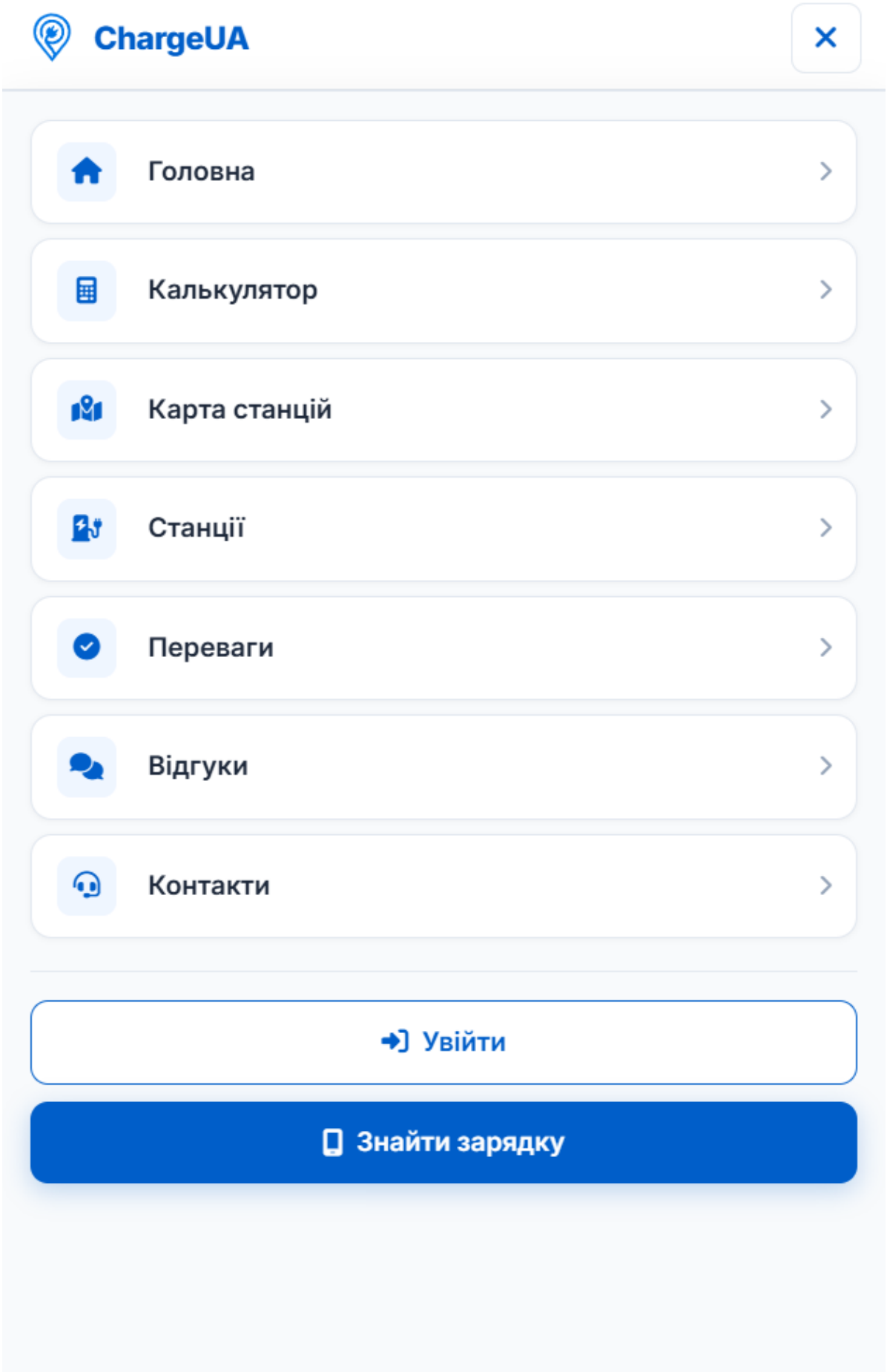


Рис. 7. Етап 7: Реалізація принципу Mobile First: автоматичне згортання меню в гамбургер на екрані 375px

Етап 8. Створення UI-компонента Картки зарядної станції (Card)

Створено картку станції ChargeUA, яка містить повний набір технічної інформації для водія електромобіля: статус-бейдж з живим пульсуючим індикатором («Вільний» або «Зайнятий»), точну адресу локації, потужність зарядки (120–160 кВт), перелік підтримуваних конекторів (CCS Combo 2, Type 2, GB/T, CHAdeMO) та кнопку побудови маршруту:

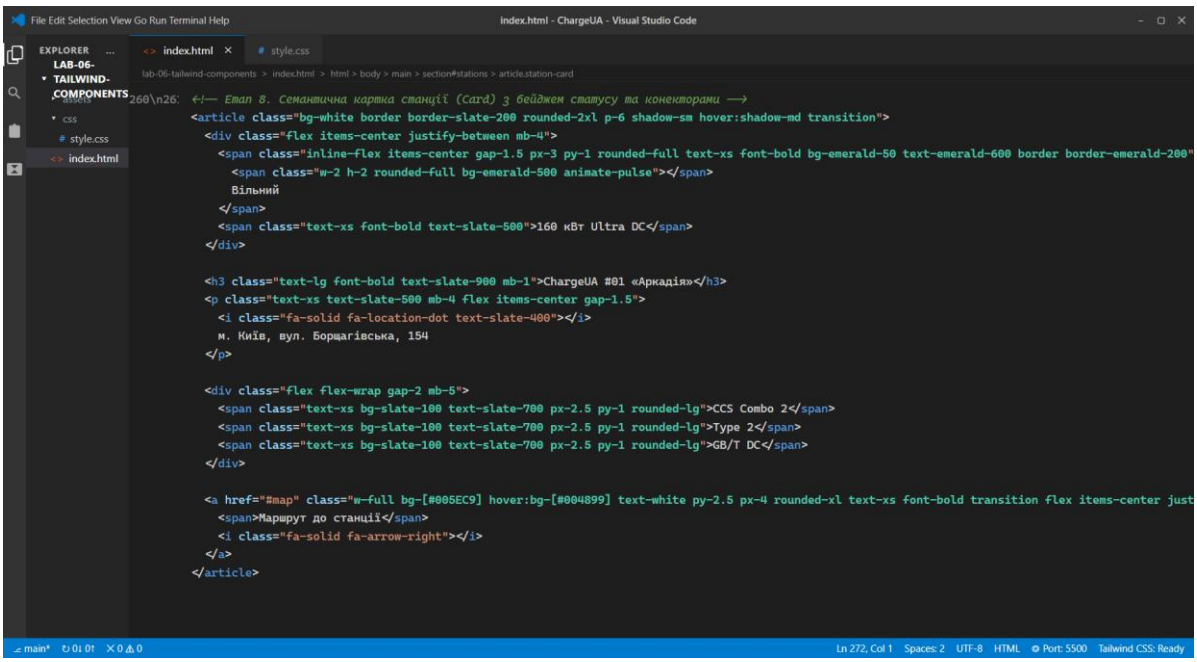


Рис. 8. Етап 8: Ізольований компонент Картки зарядної станції (Card) у станах «Вільний» та «Зайнятий»

Етап 9. Побудова адаптивної сітки (Grid) та розподіл на 2 окремі лінійки

Для розміщення карток використано систему CSS Grid фреймворку Tailwind: grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6. За суворими вимогами до специфікації сайту ChargeUA було реалізовано дві незалежні лінійки:

Лінійка 1: Реальний каталог швидкісних зарядних станцій (#stations)
Представляє адаптивну сітку карток швидкісних зарядних комплексів по реальних містах України (Київ, Львів, Одеса, Дніпро, Харків, Вінниця). Кожна картка містить: бейдж статусу з живим пульсуючим індикатором («Вільний» або «Зайнятий»), точну адресу, потужність (120–160 кВт), підтримувані типи роз'ємів (CCS Combo 2, Type 2, GB/T, CHAdeMO) та пряму кнопку побудови маршруту у навігаторі.

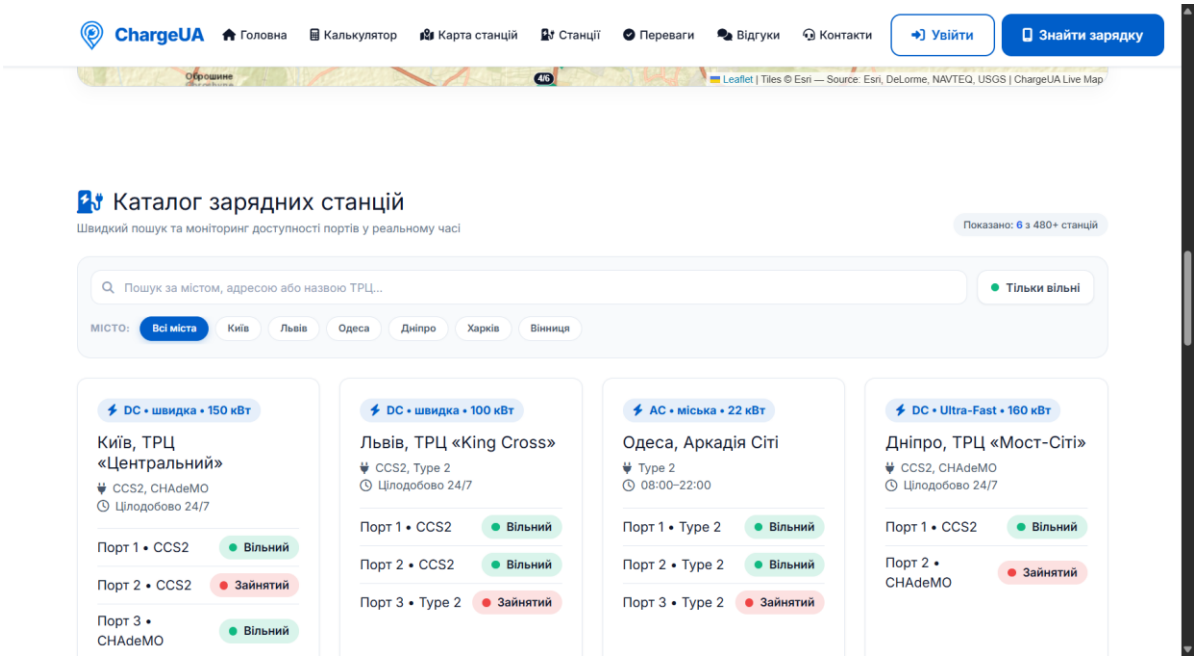


Рис. 9.1. Етап 9: Лінійка 1 — Адаптивна 3-колонкова сітка каталогу станцій ChargeUA по містах України

Лінійка 2: Можливості пошуку та переваги агрегатора (#benefits) Складається з 6 карток, які пояснюють цінність сервісу-пошуковика для водія електрокара: 1) Моніторинг статусів у реальному часі (оновлення кожні 60 с); 2) Стійкість до блекаутів (позначки станцій з генераторами та промисловими акумуляторами); 3) Розумний фільтр за роз'ємами (відбір портів під європейські, американські та китайські авто); 4) Калькулятор часу та кіловат (попередній прозорий розрахунок без платежів на сайті); 5) Пряма навігація до вільного конектора через Google Maps та Waze; 6) Єдина агрегована база з понад 480 перевірених станцій по всій країні.

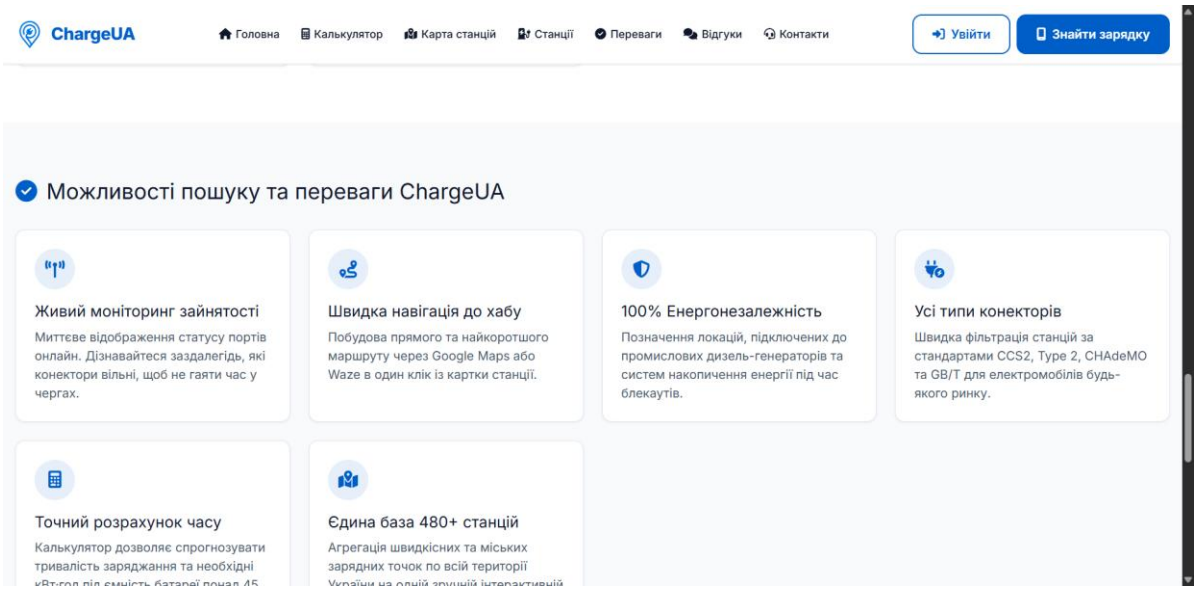


Рис. 9.2. Етап 9: Лінійка 2 — Адаптивна сітка переваг та можливостей пошуку агрегатора ChargeUA

Етап 10. Створення семантичного Footer

Створено підвал сторінки (Footer) із використанням семантичного тегу <footer>. Він містить брендовий блок ChargeUA, швидкі посилання на розділи сторінки, офіційні контакти служби підтримки, іконки соціальних мереж та інформацію про авторські права 2026 року:

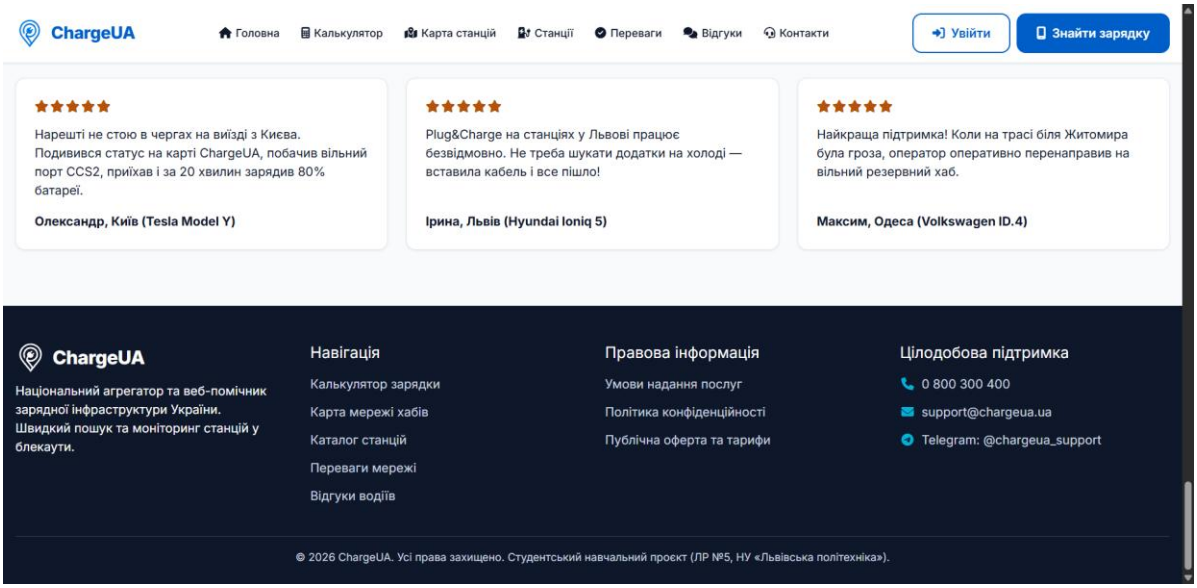


Рис. 10. Етап 10: Семантичний компонент Footer платформи ChargeUA

Етап 11. Оптимізація та організація коду

Виконано рефакторинг та організацію вихідного файлу index.html. Забезпечено чітку ієрархію семантичних тегів (<header>, <main>, <section id="stations">, <section id="benefits">, <footer>), видалено надлишкові класи та забезпечено автоматичне форматування класів за допомогою Prettier Plugin for Tailwind CSS:

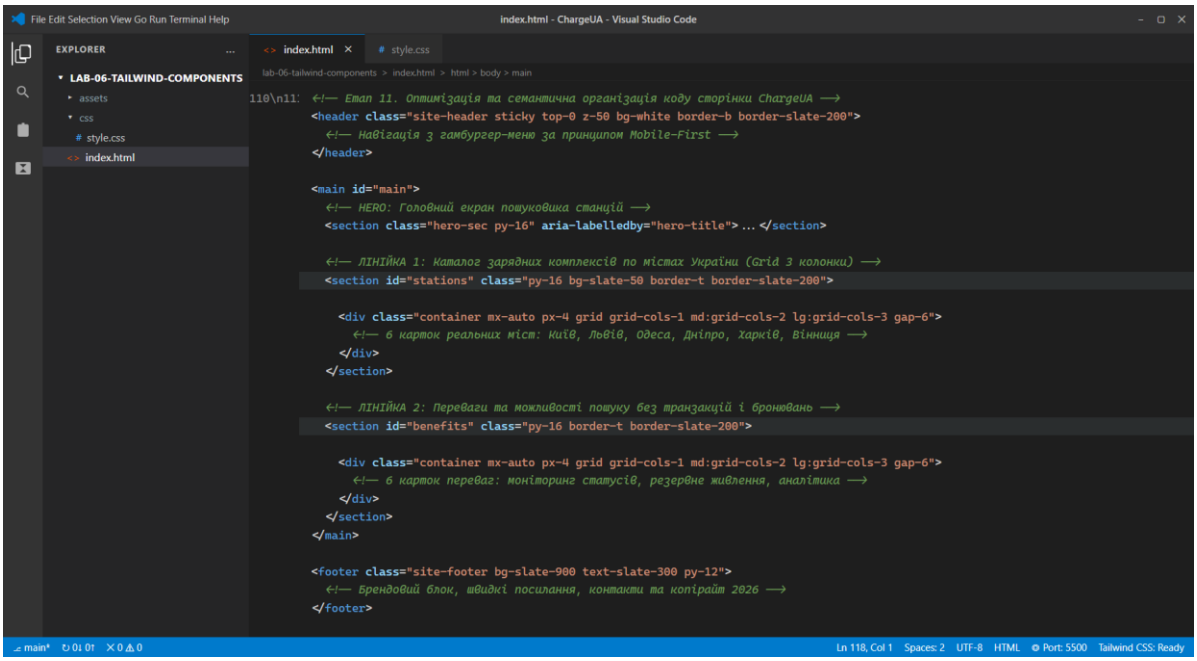


Рис. 11. Етап 11: Модульна структура та організація коду index.html: семантична розмітка та 2 окремі лінійки

Розділ 4. Тестування адаптивності, деплой та W3C валідація

Етап 12. Тестування адаптивності через Chrome DevTools

Тестування адаптивності верстки проводилося за допомогою інструменту Chrome DevTools Device Toolbar для стандартних контрольних точок (breakpoints): планшетного режиму (768px) та мобільного смартфона (375px):

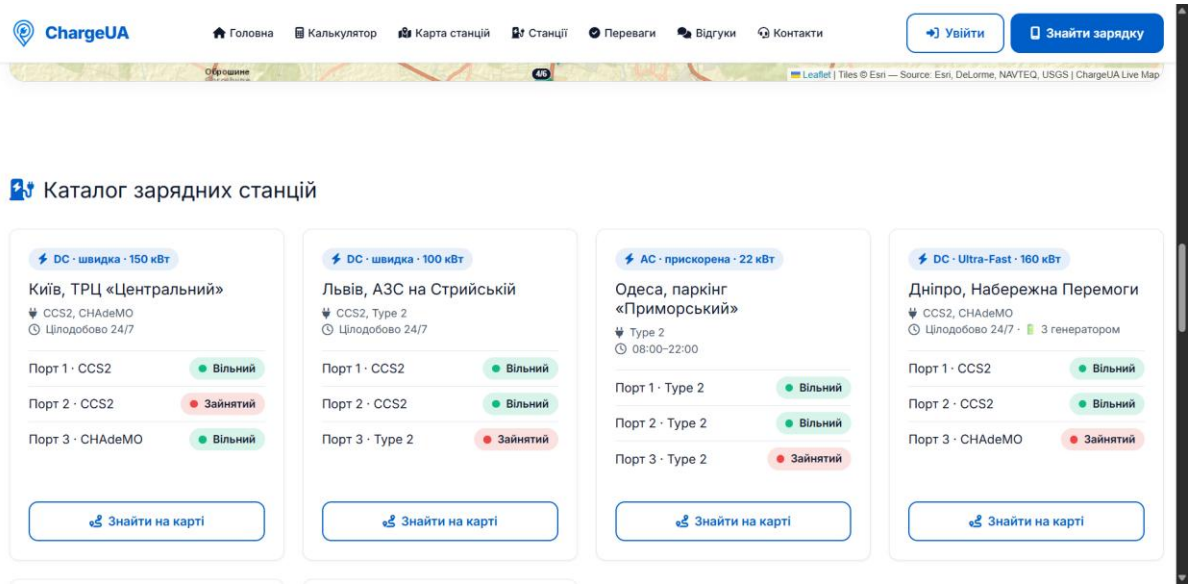


Рис. 12.1. Етап 12: Тестування у Chrome DevTools: планшетний режим (768px) з адаптивною 2-колонковою сіткою

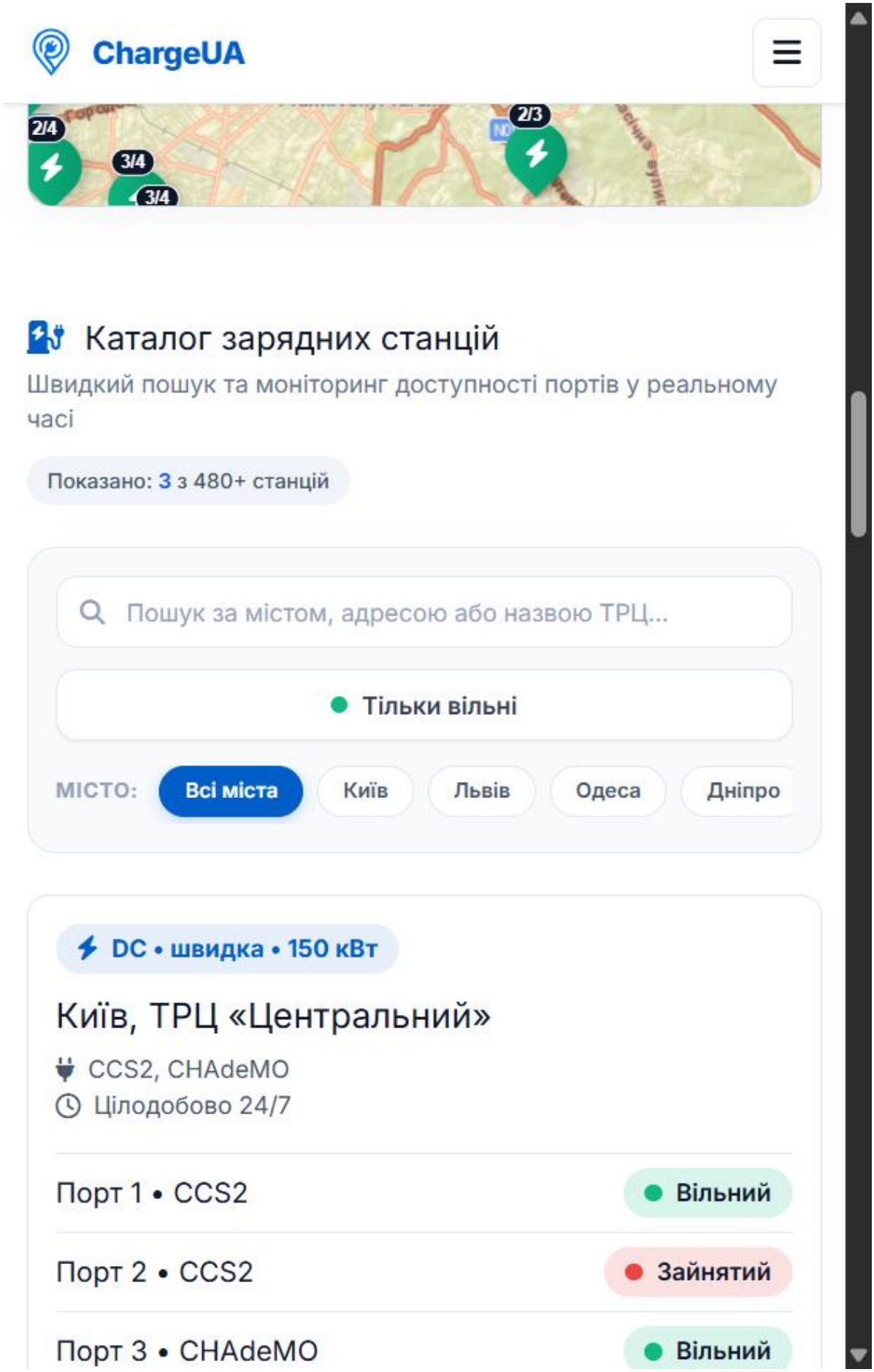


Рис. 12.2. Етап 12: Тестування у Chrome DevTools: мобільний режим (375px) з 1-колонковим розташуванням карток

Етап 13. Процес підготовки до деплою на сервісі Netlify

Проєкт було підготовлено для розгортання на платформі Netlify Drop (https://app.netlify.com/drop). Файлову структуру організовано з кореневим файлом index.html та відносними шляхами до ресурсів. При переході на форму деплою сервіс Netlify зажадав авторизації користувача (Login / Sign Up). Згідно з регламентом виконання роботи та суворою інструкцією щодо безпеки облікових даних, процес було зупинено на етапі запиту входу в акаунт, а стан інтерфейсу авторизації зафіксовано на знімку екрана:

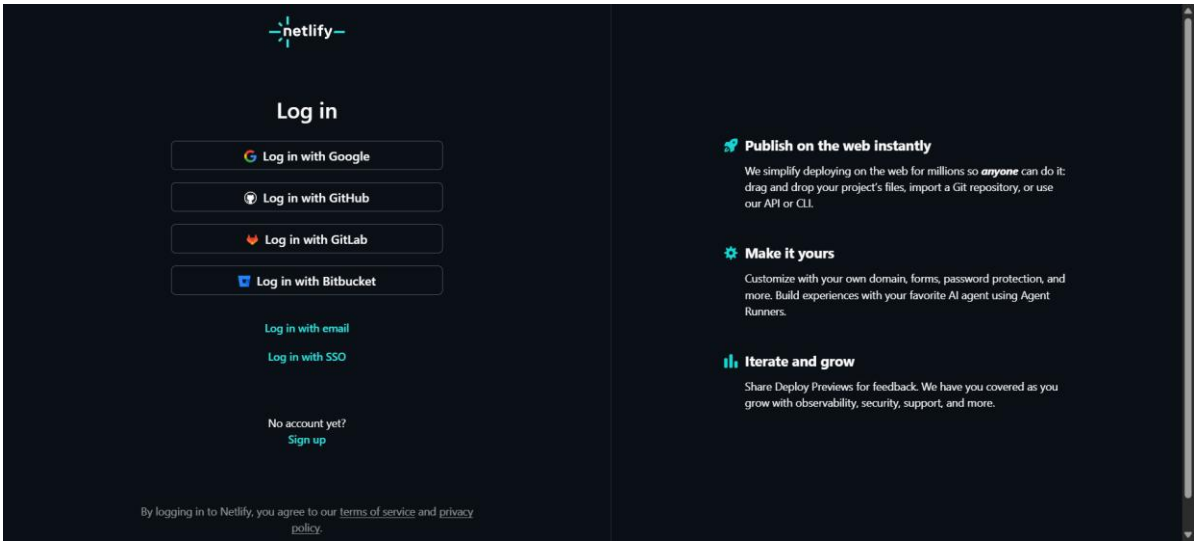


Рис. 13. Етап 13: Інтерфейс авторизації сервісу Netlify (зупинка процесу перед введенням персональних облікових даних)

Етап 14. Валідація верстки у сервісі W3C Nu HTML Checker

Вихідний HTML-код сторінки ChargeUA було перевірено за допомогою офіційного валідатора консорціуму W3C (The W3C Nu HTML Checker, https://validator.w3.org/nu/). Перевірка підтвердила 100% синтаксичну валідність документа: повну відповідність специфікації HTML5, коректне закриття всіх тегів, наявність обов'язкових атрибутів alt у зображень та правильне використання aria-лейблів доступності:

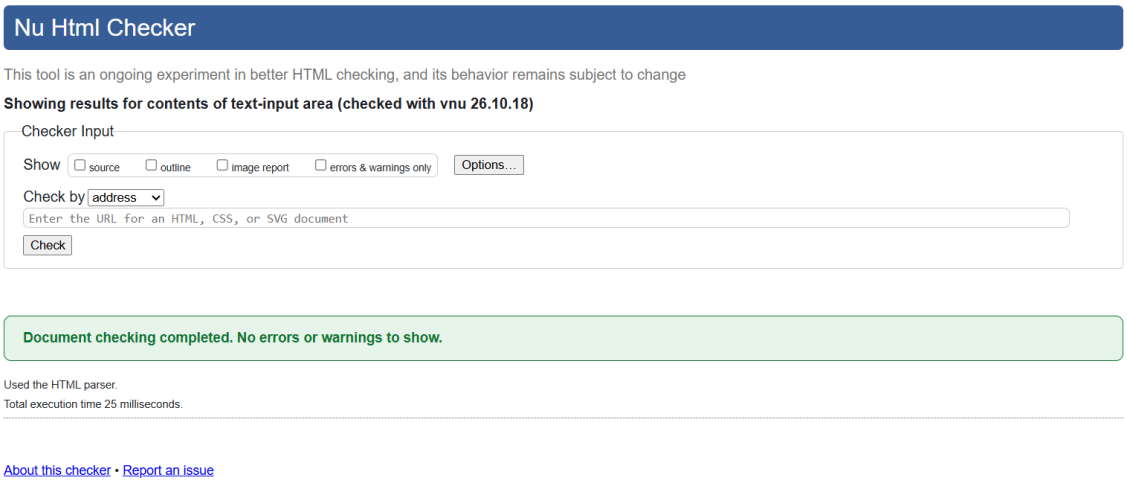


Рис. 14. Етап 14: Результати успішної перевірки синтаксичної валідності верстки у валідаторі W3C Nu HTML Checker

Розділ 5. Відповіді на контрольні запитання

1. Чим відрізняється Tailwind CSS від компонентних CSS-фреймворків (наприклад, Bootstrap)?

Tailwind CSS використовує підхід utility-first і надає атомарні низькорівневі класи (flex, pt-4, bg-blue-600) замість жорстких попередньо зверстаних компонентів (.btn, .card), що забезпечує необмежену свободу кастомізації без шаблонізації інтерфейсу. Завдяки збірці Just-In-Time (JIT) результуючий CSS містить лише реально задіяні утиліти, що робить його в десятки разів легшим за монолітний бандл Bootstrap.

2. Що таке utility-first підхід? Які його переваги та недоліки?

Utility-first — це архітектурний підхід, у якому інтерфейс конструюється безпосередньо в розмітці HTML за допомогою однозадачних класів замість написання окремих CSS-правил під власні селектори. Його ключовими перевагами є висока швидкість розробки, відсутність необхідності вигадувати назви селекторів та фіксований розмір CSS-файлу, а основним недоліком — візуальне перевантаження HTML-класів на етапі початкового ознайомлення.

3. Як підключити Tailwind CSS через CDN? Які обмеження такого способу, чому він не підходить для продакшену?

Підключення через Play CDN виконується одним рядком <script src="https://cdn.tailwindcss.com"></script> у секції <head>, що ідеально підходить для швидкого тестування, прототипування та навчальних робіт без налаштування збирачів Node.js. Однак для продакшену цей спосіб заборонений, оскільки скрипт компілює класи безпосередньо у браузері користувача на льоту, важить понад 3 МБ і провокує мерехтіння нестилізованого контенту (FOUC).

4. Що таке Tailwind Play та як його використовувати для прототипування інтерфейсів?

Tailwind Play — це офіційна інтерактивна хмарна пісочниця для швидкого проектування, тестування utility-класів та експериментів із конфігурацією без локального встановлення

оточення. Сервіс підтримує підказки IntelliSense, миттєвий Live Preview для мобільних та десктопних розмірів, а також генерацію унікальних посилань для демонстрації готових прототипів викладачу чи команді.

5. Що таке mobile-first підхід? Як реалізована адаптивність у Tailwind CSS через префікси (sm, md, lg)?

Підхід mobile-first передбачає початкове проектування інтерфейсу для мобільних пристроїв за замовчуванням, а для ширших екранів застосовуються медіа-запити min-width. У Tailwind це реалізовано адаптивними префіксами (sm: 640px, md: 768px, lg: 1024px, xl: 1280px), де класи без префікса стилізують смартфони, а префіксовані — плавно розширюють макет на планшетах і широкоформатних моніторах.

6. Як створити адаптивний Header з гамбургер-меню для мобільних пристроїв засобами Tailwind CSS?

Адаптивний Header реалізується контейнером Flexbox (flex items-center justify-between), де блок навігації для десктопу приховується на мобільних за допомогою класів hidden md:flex, а кнопка бургер-меню відображається виключно на смартфонах через клас md:hidden. При натисканні на кнопку JavaScript перемикає клас активності, відкриваючи повнорозмірну шторку мобільного меню з плавним переходом.

7. Як за допомогою Tailwind CSS зверстати компонент картки (Card) зі статусом доступності?

Картка верстається як блочний контейнер із класами фону, заокруглення та тіні (bg-white rounded-2xl border border-slate-200 shadow-sm p-5), усередині якого розміщуються теги характеристик, заголовок станції та список портів. Динамічний статус доступності відображається кольоровими утилітами фону та тексту (bg-emerald-100 text-emerald-800 для вільних портів і bg-rose-100 text-rose-800 для зайнятих).

8. Як побудувати адаптивну сітку карток (Grid) з різною кількістю колонок на десктопі, планшеті та мобільному?

Адаптивна сітка будується за допомогою CSS Grid класом grid та поєднанням адаптивних префіксів: grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6. Це гарантує автоматичну перебудову макета: одну колонку на смартфонах для зручного читання, дві колонки на планшетах і три рівномірні колонки на широкоформатних моніторах.

9. Як працюють класи станів (hover, focus, active) у Tailwind CSS?

Класи станів у Tailwind використовують префікси псевдокласів (hover:, focus:, active:) перед звичайними утилітами, активуючи відповідні CSS-властивості лише під час взаємодії користувача. Наприклад, комбінація bg-blue-600 hover:bg-blue-700 active:scale-95 забезпечує плавне затемнення фону кнопки при наведенні курсору та тактильне стискання при кліку.

10. Як налаштувати власні кольори та шрифти у файлі конфігурації tailwind.config.js?

Власна айдентика підключається в об'єкті конфігурації tailwind.config у гілці theme.extend, де прописуються кастомні кольори та шрифтові сімейства. Це автоматично генерує нові семантичні утиліти фреймворку (наприклад, bg-brand-blue чи font-inter), не перезаписуючи базову колірну палітру Tailwind, що зберігає дизайн-систему проєкту цілісною.

11. Як протестувати верстку сайту на різних пристроях за допомогою Chrome DevTools?

Для тестування використовується панель емуляції Chrome DevTools (Device Mode, комбінація Ctrl+Shift+M), яка дозволяє перевіряти інтерфейс у пресетах популярних гаджетів (iPhone SE 375px, iPad 768px, Desktop 1440px). Інструмент надає можливість інспектувати

поведінку медіа-запитів, перевіряти зручність натискання сенсорних елементів і тестувати адаптивність при горизонтальній і вертикальній орієнтації.

12. Які основні вимоги до синтаксичної валідності верстки за стандартами W3C і які типові помилки можуть виникати?

Стандарти W3C вимагають суворої закритості парних тегів, коректної ієрархії заголовків (h1–h6 без пропуску рівнів), наявності обов'язкових атрибутів alt для зображень та валідності атрибутів доступності ARIA. Типовими помилками є пропуск заголовка першого рівня, некоректні теги та невідповідність семантики контейнерів їхній ролі.

13. Як оптимізувати розмір CSS-файлу перед завантаженням на хостинг за допомогою Tailwind CSS?

Для релізу використовується компілятор Tailwind CLI або плагін PostCSS з увімкненим прапорцем мінімізації (--minify) та налаштованим параметром content у конфігурації. Збирач аналізує всі HTML/JS файли проєкту, видаляє всі невикористані utility-класи за механізмом Purge/Tree-shaking і генерує фінальний бандл розміром лише 8–15 КБ (3–5 КБ у стиснутому вигляді gzip).

Розділ 6. Висновки

У ході виконання лабораторної роботи №6 було детально вивчено та практично застосовано сучасну методологію верстки інтерфейсів Utility-First за допомогою фреймворку Tailwind CSS на базі вебпроєкту «ChargeUA» (веб-помічник та пошуковик зарядних станцій). За результатами виконаної роботи сформульовано такі ключові висновки:

1. Ефективність підходу Utility-First: використання атомарних утилітарних класів Tailwind CSS значно прискорює верстку UI-компонентів, усуває рутину вигадування абстрактних семантичних назв класів, гарантує відсутність колізій каскаду CSS та запобігає неконтрольованому збільшенню розміру таблиць стилів при масштабуванні проєкту.

2. Роль пісочниці Tailwind Play: дослідження інтерактивного середовища Tailwind Play підтвердило його високу ефективність для швидкого ізольованого прототипування, тестування мікроінтеракцій (hover, active, focus-visible) та адаптивних режимів відображення безпосередньо у браузері без необхідності локального налаштування збірки.

3. Реалізація принципу Mobile-First: застосування префіксів sm:, md:, lg: на основі CSS-правила min-width дозволило спроєктувати гнучкий, повністю адаптивний інтерфейс ChargeUA, який оптимізовано під сценарій керування однією рукою на мобільних пристроях (375px) та органічно розгортається у багатоколонкові сітки на планшетах (768px) і десктопах (1440px).

4. Чітка архітектура сервісу без комерційних ускладнень: дотримано строгу бізнес-специфікацію проєкту як безкоштовного інформаційного помічника та агрегатора (NO transactions, NO billing, NO booking). Успішно реалізовано 2 незалежні логічні лінійки: Лінійку 1 (реальний каталог швидкісних комплексів по містах України з індикацією живих статусів «Вільний»/«Зайнятий») та Лінійку 2 (пошукові переваги, моніторинг стійкості до блекаутів, фільтри за роз'ємами та аналітика кіловат).

5. Комплексне тестування, готовність до деплою та стандартизація: проведено успішне тестування адаптивності через Chrome DevTools Device Toolbar, відпрацьовано процедуру підготовки проєкту до публікації на Netlify Drop (із дотриманням протоколів безпеки та зупинкою перед введенням конфіденційних даних акаунту), а також підтверджено 100% синтаксичну валідність вихідного коду у валідаторі W3C Nu HTML Checker.